



EMBEDDED SOFTWARE ENGINEERING

Session 4 – Object-Oriented Programming in C

Laurent Fiack – laurent.fiack@ensea.fr

Bureau D212

Today

- Last time: `app.c` talks to hardware only through `bsp_t` – pure encapsulation, though we never called it that
- Today: three OOP ideas from your Java class, translated into plain C, applied to the menu we're about to build
- We'll hit a real design problem live – more than one kind of menu row – and solve it twice: by hand, then with a `vtable`

<https://youtu.be/dSLodtKuung>

Today

- 1 OOP in three concepts
- 2 OOP in C
- 3 Step 1: One Entry Type
- 4 The Problem
- 5 Step 2: Adding Polymorphism
- 6 The Vtable

New in the BSP: encoder click and OLED

- Same shape as td3 – bsp_t just grew a few more function pointers

```
int32_t bsp_encoder_get_delta(bsp_t * bsp);  
bool bsp_enc_pressed(bsp_t * bsp);  
  
void bsp_oled_clear(bsp_t * bsp);  
void bsp_oled_fill_rect(bsp_t * bsp, int x, int y, int w, int h, bool on);  
void bsp_oled_draw_string(bsp_t * bsp, int x, int y, const char *str, bool on);  
void bsp_oled_show(bsp_t * bsp);
```

- bsp_enc_pressed: same get-and-clear pattern as sw1_pressed/sw2_pressed – a click, not a level
- The OLED functions wrap an SSD1306 driver reused as-is

Exercise

- 1 Test the OLED in app_init
- 2 When the OLED works, test the encoder :
 - Write a string that changes with encoder's delta
- 3 Test the encoder push button, just to be sure

OOP in three concepts

You met these in Java as `class`, `extends`, `implements`.

- **Encapsulation:** group a state together with the operations that touch it; hide the internals behind that interface
- **Inheritance:** a type shares the shape of a more general type, and adds to it or redefines part of it
- **Polymorphism:** call the same operation on objects of different types, each reacting its own way – the caller doesn't need to know which

Where this is going

C has none of this built in. Every bit of it, we build by hand – which is exactly why it's worth seeing once.

Encapsulation, defined

- The ability to package data together with the functions that operate on it, into one unit – a *class*, in languages that have the keyword
- That unit exposes an **interface** (what you can do with it) and hides an **implementation** (how it actually does it) – callers only ever need the interface
- In C: no keyword, but the same split maps directly onto files – a `.h` declares the interface, the matching `.c` hides the implementation

A small example, before we come back to the project

A Shape: a position, and a couple of operations on it. Nothing to do with the menu – just encapsulation, isolated.

Example: Shape's interface (shape.h)

```
#ifndef SHAPE_H
#define SHAPE_H

#include <stdint.h>

/* Shape's attributes... */
typedef struct {
    int16_t x; /* x-coordinate of Shape's position */
    int16_t y; /* y-coordinate of Shape's position */
} Shape;

/* Shape's operations... */
void Shape_ctor(Shape * const me, int16_t x0, int16_t y0);
void Shape_moveBy(Shape * const me, int16_t dx, int16_t dy);
uint16_t Shape_distanceFrom(Shape const * const me,
                             Shape const * other);

#endif /* SHAPE_H */
```

- Everything a caller needs to know: the struct's shape, and the three operations – nothing here says *how* they work

Example: Shape's implementation (1/2)

```
#include "shape.h" /* Shape class interface */

void Shape_ctor(Shape * const me, int16_t x0, int16_t y0) {
    me->x = x0;
    me->y = y0;
}

void Shape_moveBy(Shape * const me, int16_t dx, int16_t dy) {
    me->x += dx;
    me->y += dy;
}
```

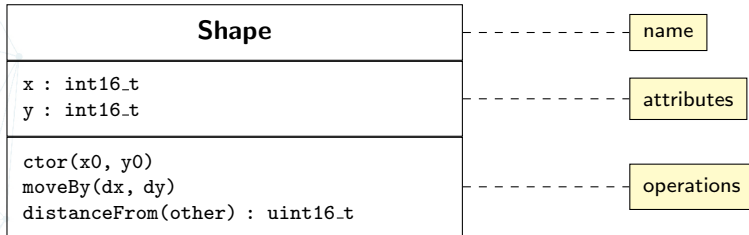
- `me`: the object being operated on, passed explicitly – exactly the `bsp_t *bsp` pattern from session 3, just with a different name
- `Shape * const me`: a *const pointer* (can't be reassigned) to a *non-const* `Shape` (its fields can be written) – these two operations need to write `x/y`

Example: Shape's implementation (2/2)

```
uint16_t Shape_distanceFrom(Shape const * const me,  
                             Shape const * other)  
{  
    int16_t dx = me->x - other->x;  
    int16_t dy = me->y - other->y;  
    if (dx < 0) {  
        dx = -dx;  
    }  
    if (dy < 0) {  
        dy = -dy;  
    }  
    return dx + dy;  
}
```

- This one only reads – so me is Shape const * const: const pointer to const data, nothing here can write through it
- Manhattan distance, on purpose: keeps it integer-only, no `<math.h>` needed for this example

Shape as a UML class



- Three compartments: class name, attributes (the struct's fields, grouped in one box), operations (the interface's functions, grouped in one box)
- Language-agnostic on purpose: this box describes Shape whether it ends up as a C struct+functions, a Java class, or anything else

Instantiating: static, automatic, dynamic

```
Shape s1; /* static allocation */

int main(void) {
    Shape s2; /* automatic allocation */
    Shape *ps3 = malloc(sizeof(Shape)); /* dynamic allocation */

    Shape_ctor(&s1, 1, 2);
    Shape_ctor(&s2, 3, 4);
    Shape_ctor(ps3, 5, 6);

    Shape_moveBy(&s1, 7, 8);
    Shape_moveBy(&s2, 9, 10);
    Shape_moveBy(ps3, -1, -2);

    free(ps3);
}
```

- An **object** is an instance of a class – s1, s2, *ps3 are three Shape instances, living in three different places (static storage, the stack, the heap)
- Same Shape_ctor/Shape_moveBy calls work on all three – the operations don't care how the object was allocated, only that they got a valid pointer to one

Previous examples already does that

- A struct, plus functions that take a pointer to it as their first parameter
- You've been doing this since session 3 – just never under this name

```
typedef struct { /* ... */ } bsp_t;
void bsp_led_set(bsp_t * bsp, uint8_t led, bool on);
```

```
typedef struct { bsp_t bsp; /* ... */ } app_t;
void app_process(app_t *app);
```

- The state (bsp_t, app_t) and the operations on it travel together – encapsulation, exactly as defined a moment ago

Inheritance, defined

- A type shares the shape of a more general type, and adds to it or redefines part of it
- In C, no keyword either: embed the base struct as the derived struct's *first field* – not a pointer, the actual base object, inline
- Because it's first, a pointer to the derived struct and a pointer to that embedded base struct share the same address – casting between the two is always valid

Back to Shape

A Rectangle *is a* Shape, plus a width and a height. Same running example, extended.

Example: Rectangle's interface

```

/* Rectangle's attributes... */
typedef struct {
    Shape super; /* inherited Shape */

    /* attributes specific to this class */
    uint16_t width;
    uint16_t height;
} Rectangle;

/* Rectangle's operations... */
void Rectangle_ctor(Rectangle * const me,
                    int16_t x0, int16_t y0,
                    uint16_t w0, uint16_t h0);
void Rectangle_draw(Rectangle const * const me);
uint32_t Rectangle_area(Rectangle const * const me);

```

- Shape super as the very first field: Rectangle *is* a Shape, with width/height bolted on
- Every Shape operation (Shape_moveBy, Shape_distanceFrom, ...) already works on a Rectangle – give it &r->super

Example: Rectangle's implementation

```
void Rectangle_ctor(Rectangle * const me,
                    int16_t x0, int16_t y0,
                    uint16_t w0, uint16_t h0)
{
    Shape_ctor(&me->super, x0, y0); /* base class ctor */

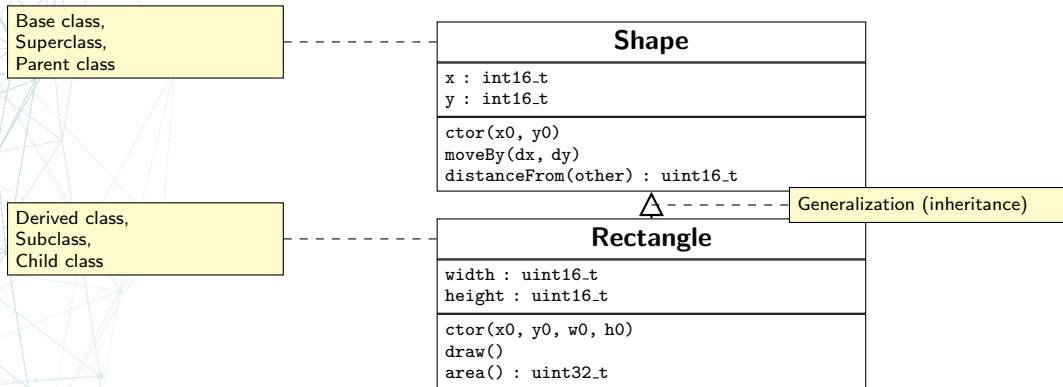
    /* init attributes added in this class */
    me->width = w0;
    me->height = h0;
}

void Rectangle_draw(Rectangle const * const me) {
    // bsp draw...
}

uint32_t Rectangle_area(Rectangle const * const me) {
    return (uint32_t)me->width * (uint32_t)me->height;
}
```

- Rectangle_ctor builds the base part first (Shape_ctor on &me->super), then the specific part – same discipline as a super(...) call in Java

Shape / Rectangle as UML classes



- Hollow triangle, derived → base: *generalization*; the reverse reading is *specialization*
- Two names, same relationship: base/super/parent class vs. derived/sub/child class

Instantiating a Rectangle

```
Rectangle r1;  
uint32_t a;  
  
int main(void) {  
    Rectangle_ctor(&r1, 1, 2, 15, 10);  
  
    Rectangle_draw(&r1);  
    a = Rectangle_area(&r1);  
    Shape_moveBy((Shape *)&r1, 7, 8);  
}
```

- `(Shape *)&r1`: valid because super is Rectangle's first field – a Rectangle's address *is* its embedded Shape's address
- `Shape_moveBy` has no idea it's touching a Rectangle – it moves x/y, exactly as it would for a plain Shape

Polymorphism, the easy way

- Polymorphism: call the same operation on objects of different concrete types, each reacting its own way
 - The caller never needs to know which
- The simplest way to get there in C:
 - Give the *base* type a tag saying which concrete type this really is,
 - And write one dispatch function per operation that reacts to it
- The tag lives in Shape itself, the part every concrete type shares
- Same for the dispatch function : calls the concrete type's operation

Coming up

Two concrete shapes, Rectangle and Circle, both *are* a Shape – Shape_draw/Shape_area become the one call site for either.

Shape's interface, revisited

```
typedef enum { SHAPE_RECTANGLE, SHAPE_CIRCLE } shape_kind_t;

typedef struct {
    shape_kind_t kind;    /* NEW: which concrete type is this? */
    int16_t x, y;
} Shape;

void Shape_ctor(Shape * const me, shape_kind_t kind, int16_t x0, int16_t y0);
void Shape_moveBy(Shape * const me, int16_t dx, int16_t dy);
uint16_t Shape_distanceFrom(Shape const * const me, Shape const * other);

/* NEW: one call site, dispatches on kind -- see the implementation later */
void Shape_draw(Shape const * const me);
uint32_t Shape_area(Shape const * const me);
```

- Shape_ctor grows one parameter: every constructor now says which concrete type it's building – Rectangle_ctor's call becomes Shape_ctor(&me->super, SHAPE_RECTANGLE, x0, y0)
- Shape_draw/Shape_area: declared once here, implemented once – callers never call Rectangle_draw/Circle_draw directly again

Example: Circle's interface

```
/* Circle's attributes... */
typedef struct {
    Shape super; /* inherited Shape */

    /* attribute specific to this class */
    uint16_t radius;
} Circle;

/* Circle's operations... */
void Circle_ctor(Circle * const me,
                int16_t x0, int16_t y0,
                uint16_t radius0);
void Circle_draw(Circle const * const me);
uint32_t Circle_area(Circle const * const me);
```

- Same shape as Rectangle: Shape super first, one attribute of its own
- Circle_draw/Circle_area still exist, but they're no longer called from outside shape.c – only Shape_draw/Shape_area are, from inside the switch

Implementation: Shape_draw/Shape_area, the switch

```

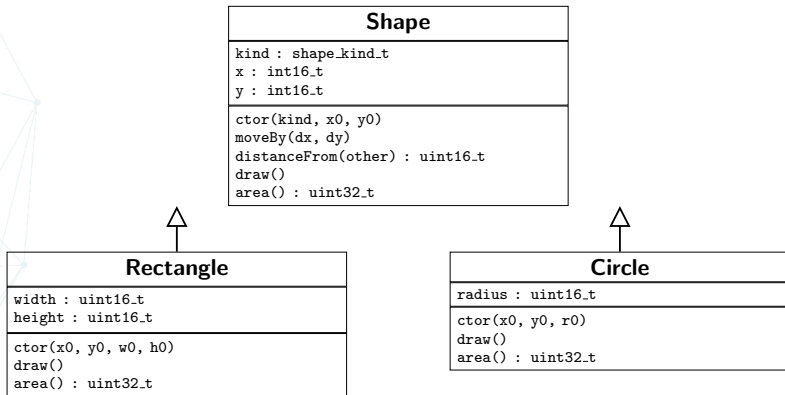
void Shape_draw(Shape const * const me) {
    switch (me->kind) {
        case SHAPE_RECTANGLE:
            Rectangle_draw((Rectangle const *)me);
            break;
        case SHAPE_CIRCLE:
            Circle_draw((Circle const *)me);
            break;
    }
}

uint32_t Shape_area(Shape const * const me) {
    switch (me->kind) {
        case SHAPE_RECTANGLE: return Rectangle_area((Rectangle const *)me);
        case SHAPE_CIRCLE:    return Circle_area((Circle const *)me);
    }
    return 0;
}

```

- (Rectangle const *)me: the same downcast as Shape_moveBy((Shape *)&r1, ...) earlier, just the other direction – valid because super is always the first field
- Exactly one place in the whole program that knows both concrete types exist – neither Rectangle_draw nor Circle_draw needs to know the other is out there

Shape, Rectangle, Circle as UML classes



- Two arrows into the same base: both *Rectangle* and *Circle* are a *Shape*
- `draw()`/`area()` now live on *Shape* too – the polymorphic entry point, dispatching to whichever concrete implementation actually runs

Example: Circle's implementation

```
void Circle_ctor(Circle * const me,
                int16_t x0, int16_t y0,
                uint16_t radius0)
{
    Shape_ctor(&me->super, SHAPE_CIRCLE, x0, y0); /* base class ctor */

    me->radius = radius0;
}

void Circle_draw(Circle const * const me) {
    // bsp draw...
}

uint32_t Circle_area(Circle const * const me) {
    return (uint32_t)(3 * me->radius * me->radius); /* pi ≈ 3, integer-only */
}
```

- Same discipline as Rectangle_ctor: base part first – now passing SHAPE_CIRCLE, so Shape_draw/Shape_area know how to dispatch back to this one
- $\pi \approx 3$, on purpose: integer-only, same shortcut as Shape_distanceFrom's Manhattan distance earlier

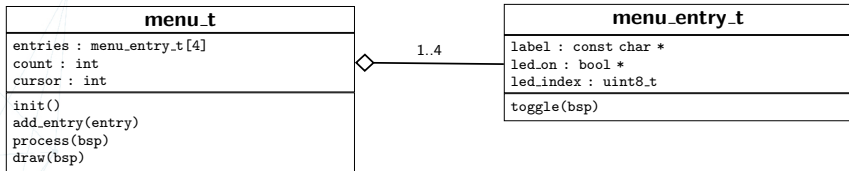
Step 1: the goal

- Build a menu, displayed on the OLED and driven by the encoder, that lets you switch the LEDs on and off
- Rotate the encoder: move the cursor. Click: toggle the selected LED

> LED1: ON
LED2: OFF
LED3: OFF

- For now, only one kind of row: an LED toggle

Step 1: UML



- A `menu_t` *contains* up to four `menu_entry_t`: composition, not inheritance

Step 1: plumbing

- Create a `menu.c/menu.h` pair in the `app` folder
- Add `menu.c` to the sources in `CMakeLists.txt`
`app/menu.c`
- Everything menu-related goes in `menu.c/menu.h`, entries included – fewer files to juggle during the lab

Start with just the LEDs

```
typedef struct {  
    const char *label;  
    bool *led_on;  
    uint8_t led_index;  
} menu_entry_t;  
  
void menu_entry_toggle(menu_entry_t *e, const bsp_t *bsp) {  
    *e->led_on = !*e->led_on;  
    bsp->led_set(bsp->ctx, e->led_index, *e->led_on);  
}
```

- menu_t holds an array of these, a cursor; navigating calls menu_entry_toggle on click, draws "%s: %s" for every row
- no inheritance, no polymorphism – with a single possible kind, there's nothing to distinguish yet

Putting entries in a list

```
typedef struct {  
    menu_entry_t entries[4];  
    int count;  
    int cursor;  
} menu_t;  
  
void menu_init(menu_t *m) {  
    m->count = 0;  
    m->cursor = 0;  
}  
  
void menu_add_entry(menu_t *m, menu_entry_t entry) {  
    m->entries[m->count++] = entry;  
}
```

Reacting to the encoder

```
void menu_process(menu_t *m, const bsp_t *bsp) {  
    int32_t delta = bsp->encoder_get_delta(bsp);  
    if (delta != 0) {  
        m->cursor += (int)delta;  
        if (m->cursor < 0)          { m->cursor = 0; }  
        if (m->cursor >= m->count) { m->cursor = m->count - 1; }  
    }  
    if (bsp->encoder_button_pressed(bsp)) {  
        menu_entry_toggle(&m->entries[m->cursor], bsp);  
    }  
}
```

- Rotate: move the cursor, clamped to the list. Click: toggle whichever row is selected.

Drawing it

```
void menu_draw(const menu_t *m, const bsp_t *bsp) {
    bsp->oled_clear(bsp);

    for (int row = 0; row < m->count; row++) {
        int y = 4 + row * 15;
        bool selected = (row == m->cursor);
        if (selected) {
            bsp->oled_fill_rect(bsp, 0, y, 127, 13, true);
            bsp->oled_draw_string(bsp, 4, y + 2, ">", false);
        }

        char line[20];
        const menu_entry_t *e = &m->entries[row];
        snprintf(line, sizeof(line), "%s: %s", e->label, *e->led_on ? "ON" : "OFF");
        bsp->oled_draw_string(bsp, 16, y + 2, line, !selected);
    }

    bsp->oled_show(bsp);
}
```

Wiring it up

```
typedef struct {
    bsp_t *bsp;

    menu_t menu;
    bool led_on[3];
} app_t;

static const char *const LED_LABELS[3] = {"LED1", "LED2", "LED3"};

void app_init(app_t *app, bsp_t *bsp) {
    app->bsp = bsp;

    menu_init(&app->menu);
    for (int i = 0; i < 3; i++) {
        app->led_on[i] = false;
        menu_entry_t entry = { LED_LABELS[i], &app->led_on[i], (uint8_t)i };
        menu_add_entry(&app->menu, entry);
    }
}

void app_process(app_t *app) {
    menu_process(&app->menu, app->bsp);
    menu_draw(&app->menu, app->bsp);
}
```

- The menu and the LED states live in app_t: app.c owns them, menu.c only gets pointers
- That's everything – this compiles, flashes, and shows 3 rows you can scroll and toggle

Now we also want...

- One new entry, NEOPIXEL, that doesn't hold a value at all – it opens a second page
- In the second page : three more entries – HUE/SAT/VAL
- A way to go back to the main page

How do you extend `menu_entry_t` for that?

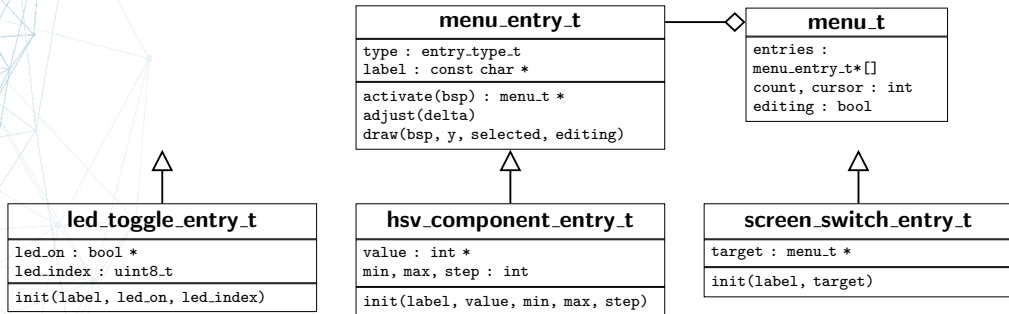
Two tempting answers – both bad

- 1 **A separate module per kind** – `led_menu.c`, `hsv_page.c`, each with its own navigate/draw loop.
 - It works. But navigate/draw logic is duplicated : danger!
- 2 **Every possible field, flat, in one struct** – `int *int_value`, `menu_t *submenu_target`, all living side by side.
 - Compiles. Nothing stops a bool row from touching the int fields by mistake, and the struct only grows from here.

Neither actually separates "what every row has in common" from "what depends on its kind"

Let's use inheritance/polymorphism!

Step 2: entries as UML classes



- **menu_t** holds pointers to the *base* type: it neither knows nor cares which concrete entries it holds

The *super* menu_entry_t

```
typedef struct menu menu_t;

typedef enum { ENTRY_BOOL, ENTRY_INT, ENTRY_SUBMENU } entry_type_t;

typedef struct {
    entry_type_t type;
    const char *label;
} menu_entry_t;
```

The *concrete* entries

```
typedef struct {  
    menu_entry_t super; /* inherited menu_entry_t */  
    bool *led_on;  
    uint8_t led_index;  
} led_toggle_entry_t;  
  
typedef struct {  
    menu_entry_t super; /* inherited menu_entry_t */  
    int *value;  
    int min;  
    int max;  
    int step;  
} hsv_component_entry_t;  
  
typedef struct {  
    menu_entry_t super; /* inherited menu_entry_t */  
    menu_t *target;  
} screen_switch_entry_t;
```

Interface

```
// Generic menu_entry operations
menu_t *menu_entry_activate(menu_entry_t *self, bsp_t *bsp);
void menu_entry_adjust(menu_entry_t *self, int32_t delta);
void menu_entry_draw(const menu_entry_t *self, bsp_t *bsp,
                     int y, bool selected, bool editing);

// Concrete entries operations
void led_toggle_entry_init(led_toggle_entry_t *e,
                          const char *label, bool *led_on, uint8_t led_index);
void hsv_component_entry_init(hsv_component_entry_t *e,
                              const char *label, int *value, int min, int max, int step);
void screen_switch_entry_init(screen_switch_entry_t *e,
                              const char *label, menu_t *target);
```

Updated menu interface

```
struct menu {  
    menu_entry_t *entries[MAX_ENTRIES];  
    int count;  
    int cursor;  
    bool editing;  
};  
  
void menu_init(menu_t *m);  
void menu_add_entry(menu_t *m, menu_entry_t *entry);  
void menu_enter(menu_t *m);  
  
menu_t *menu_process(menu_t *m, bsp_t *bsp);  
void menu_draw(const menu_t *m, bsp_t *bsp);
```

- Entries are stored by pointer to allow polymorphism
- menu_process returns the new menu

Implementation : Helper function

```
static void menu_entry_draw_row(bsp_t *bsp,
    int y, bool selected, bool editing, const char *line) {
    if (selected) {
        bsp->oled_fill_rect(bsp, 0, y, 127, 13, true);
        bsp->oled_draw_string(bsp, 4, y + 2, editing ? "*" : ">", false);
    }
    bsp->oled_draw_string(bsp, 16, y + 2, line, !selected);
}
```

Implementation : led_toggle_entry_t

```

// constructor is public
void led_toggle_entry_init(led_toggle_entry_t *e, const char *label, bool *led_on, uint8_t led_index) {
    e->super.kind = MENU_ENTRY_LED_TOGGLE;
    e->super.label = label;
    e->led_on = led_on;
    e->led_index = led_index;
}

// concrete function are hidden from the user
static void led_toggle_entry_toggle(led_toggle_entry_t *e, bsp_t *bsp) {
    *e->led_on = !*e->led_on;
    bsp->led_set(bsp, e->led_index, *e->led_on);
}

static void led_toggle_entry_draw(const led_toggle_entry_t *e, bsp_t *bsp, int y, bool selected) {
    char line[20];
    snprintf(line, sizeof(line), "%s: %s", e->super.label, *e->led_on ? "ON" : "OFF");
    menu_entry_draw_row(bsp, y, selected, false, line);
}

```

Implementation : screen_switch_entry_t

```
void screen_switch_entry_init(screen_switch_entry_t *e,  
    const char *label, menu_t *target) {  
    e->super.kind = MENU_ENTRY_SCREEN_SWITCH;  
    e->super.label = label;  
    e->target = target;  
}  
  
static void screen_switch_entry_draw(const screen_switch_entry_t *e,  
    bsp_t *bsp, int y, bool selected) {  
    menu_entry_draw_row(bsp, y, selected, false, e->super.label);  
}
```

Implementation : hsv_component_entry_t

```

void hsv_component_entry_init(hsv_component_entry_t *e, const char *label,
    int *value, int min, int max, int step) {
    e->super.kind = MENU_ENTRY_HSV_COMPONENT;
    e->super.label = label;
    e->value = value;
    e->min = min;
    e->max = max;
    e->step = step;
}

static void hsv_component_entry_adjust(hsv_component_entry_t *e, int32_t delta) {
    int value = *e->value + (int)delta * e->step;
    if (value < e->min) {
        value = e->min;
    }
    if (value > e->max) {
        value = e->max;
    }
    *e->value = value;
}

static void hsv_component_entry_draw(const hsv_component_entry_t *e, bsp_t *bsp,
    int y, bool selected, bool editing) {
    char line[20];
    snprintf(line, sizeof(line), "%s: %d", e->super.label, *e->value);
    menu_entry_draw_row(bsp, y, selected, editing, line);
}

```

Implementation : polymorphic dispatch (1/3)

```
menu_t *menu_entry_activate(menu_entry_t *self, bsp_t *bsp) {  
    switch (self->kind) {  
        case MENU_ENTRY_LED_TOGGLE:  
            led_toggle_entry_toggle((led_toggle_entry_t *)self, bsp);  
            return NULL;  
        case MENU_ENTRY_SCREEN_SWITCH:  
            return ((screen_switch_entry_t *)self)->target;  
        case MENU_ENTRY_HSV_COMPONENT:  
            return NULL; /* editable: activate() never runs, see menu_process */  
    }  
    return NULL;  
}
```

Implementation : polymorphic dispatch (2/3)

```
void menu_entry_adjust(menu_entry_t *self, int32_t delta) {  
    switch (self->kind) {  
        case MENU_ENTRY_LED_TOGGLE:  
        case MENU_ENTRY_SCREEN_SWITCH:  
            break; /* not editable, nothing to adjust */  
        case MENU_ENTRY_HSV_COMPONENT:  
            hsv_component_entry_adjust((hsv_component_entry_t *)self, delta);  
            break;  
    }  
}
```

Implementation : polymorphic dispatch (3/3)

```
void menu_entry_draw(const menu_entry_t *self, bsp_t *bsp, int y, bool selected, bool editing) {  
    switch (self->kind) {  
        case MENU_ENTRY_LED_TOGGLE:  
            led_toggle_entry_draw((const led_toggle_entry_t *)self, bsp, y, selected);  
            break;  
        case MENU_ENTRY_SCREEN_SWITCH:  
            screen_switch_entry_draw((const screen_switch_entry_t *)self, bsp, y, selected);  
            break;  
        case MENU_ENTRY_HSV_COMPONENT:  
            hsv_component_entry_draw((const hsv_component_entry_t *)self, bsp, y, selected, editing);  
            break;  
    }  
}
```

Implementation : the new process

```

menu_t *menu_process(menu_t *m, bsp_t *bsp) {
    int32_t delta = bsp->encoder_get_delta(bsp);
    menu_entry_t *entry = m->entries[m->cursor];

    if (m->editing) {
        if (delta != 0) { menu_entry_adjust(entry, delta); }
        if (bsp->enc_pressed(bsp)) { m->editing = false; }
        return NULL;
    }

    if (delta != 0) {
        m->cursor += (int)delta;
        if (m->cursor < 0) { m->cursor = 0; }
        if (m->cursor >= m->count) { m->cursor = m->count - 1; }
        entry = m->entries[m->cursor];
    }

    if (bsp->enc_pressed(bsp)) {
        if (entry->kind == MENU_ENTRY_HSV_COMPONENT) { // Editable
            m->editing = true;
            return NULL;
        }
        return menu_entry_activate(entry, bsp);
    }

    return NULL;
}

```

app interface

```
typedef struct {  
    bsp_t * bsp;  
  
    menu_t main_menu;  
    menu_t hsv_menu;  
    menu_t *active;  
  
    bool led_on[3];  
  
    int hue;  
    int sat;  
    int val;  
  
} app_t;
```

Instantiation in app : the entries

```
static led_toggle_entry_t led_entries[3];  
static screen_switch_entry_t neopixel_entry;  
  
static hsv_component_entry_t hue_entry;  
static hsv_component_entry_t sat_entry;  
static hsv_component_entry_t val_entry;  
static screen_switch_entry_t back_entry;
```

Instanciación en app : app_init

```

void app_init(app_t *app, bsp_t *bsp) {
    app->bsp = bsp;

    menu_init(&app->main_menu);
    for (int i = 0; i < 3; i++) {
        app->led_on[i] = false;
        led_toggle_entry_init(&led_entries[i], LED_LABELS[i], &app->led_on[i], (uint8_t)i);
        menu_add_entry(&app->main_menu, &led_entries[i].super);
    }
    screen_switch_entry_init(&neopixel_entry, "NEOPIXEL", &app->hsv_menu);
    menu_add_entry(&app->main_menu, &neopixel_entry.super);

    app->hue = 0;
    app->sat = 255;
    app->val = 30;
    menu_init(&app->hsv_menu);
    hsv_component_entry_init(&hue_entry, "HUE", &app->hue, 0, 359, 1);
    hsv_component_entry_init(&sat_entry, "SAT", &app->sat, 0, 255, 1);
    hsv_component_entry_init(&val_entry, "VAL", &app->val, 0, 255, 1);
    screen_switch_entry_init(&back_entry, "BACK", &app->main_menu);
    menu_add_entry(&app->hsv_menu, &hue_entry.super);
    menu_add_entry(&app->hsv_menu, &sat_entry.super);
    menu_add_entry(&app->hsv_menu, &val_entry.super);
    menu_add_entry(&app->hsv_menu, &back_entry.super);

    app->active = &app->main_menu;
}

```

Instantiation in app : app_process

```
void app_process(app_t *app) {  
    bsp_t * bsp = app->bsp;  
  
    menu_t *next = menu_process(app->active, bsp);  
    if (next != NULL) {  
        app->active = next;  
        menu_enter(app->active);  
    }  
  
    bsp->neopixel_set_hsv(bsp, app->hue, app->sat, app->val);  
  
    menu_draw(app->active, bsp);  
}
```

Polymorphism, the hard way : Vtables

- Bonus, if you're motivated enough
- That's how it's implemented in C++
- https://youtu.be/2v_qM5SJD1Y

Take away

- Encapsulation: nothing to add, you were already doing it
- Inheritance works by *owning* the super class
- Polymorphism can be implemented thanks to a dispatch function
- Polymorphism doesn't require a vtable
- Next session: this same menu, but with *event-driven programming*